# Ethical Hacking Lab Record

Name: Deepan SA

Roll Number: 241901021

**Index of Experiments**

1. Footprinting the target network using search engines, web services and social networking sites
2. Website, email, whois, DNS and network footprinting on the target network
3. Session hijacking using various tools
4. Detecting session hijacking
5. Web server reconnaissance using various tools
6. Enumerating web server information
7. Web application reconnaissance using various tools
8. Web spidering
9. Web application vulnerability scanning
10. Cracking FTP credentials using a dictionary attack
11. Performing Cross-site Request Forgery (CSRF) attack
12. Identifying XSS vulnerabilities in web applications
13. Detecting web application vulnerabilities using various web application security tools
14. Performing an SQL injection attack against MSSQL to extract databases
15. Detecting SQL injection vulnerabilities using various SQL injection detection tools
16. Footprinting a wireless network
17. Creating a rogue access point to capture data packets

# Experiment 1: Footprinting the target network using search engines, web services and social networking sites

**Aim:** To collect publicly available information about the target organisation using search engines, web services and social networking sites.

**Tools required:**

- Google search and advanced search operators
- Wayback Machine (archive.org)
- LinkedIn and Facebook (public pages)
- theHarvester
- Maltego Community Edition (optional)

**Procedure**

1 Open a browser in Kali Linux and search for the organisation name in quotes.
2 Use operators such as site:, intitle:, filetype:pdf and inurl: to find documents, login pages and public files of the target.
3 Search the Wayback Machine for old versions of the target website to find pages that are no longer linked.
4 Search social networking sites for employee names, job posts, office locations and contact details that are openly visible.
5 Run theHarvester against the target domain to collect email addresses, subdomains and host names from search engines.
6 Record every finding in a table with the source from which it was collected.
7 Note the date on which each page or document was published, since older information may no longer be valid.
8 Save the links or screenshots of each useful result so that the source can be shown later in the report.
9 Classify the collected information into categories such as people, technology, location and contact details.

**Result:** Public information such as the company address, employee names, email address format, subdomains and some older pages of the website was collected using search engines, web archives and social networking sites. The footprinting data was recorded successfully.

# Experiment 2: Website, email, whois, DNS and network footprinting on the target network

**Aim:** To gather detailed information about the target domain such as registration details, DNS records, mail servers and network range.

**Tools required:**

- whois
- nslookup and dig
- host
- traceroute
- ping
- theHarvester
- Netcraft (web-based)

**Procedure**

1 Run whois <domain> in the terminal to obtain registrar, registration dates and name server details.
2 Run nslookup <domain> and dig <domain> ANY to list the A, MX, NS and TXT records.
3 Run dig MX <domain> to identify the mail servers and dig NS <domain> to identify the name servers.
4 Run host -t mx <domain> and check the mail exchanger priority values.
5 Run traceroute <target IP> and ping <target IP> to check the network path and whether the host is reachable.
6 Use theHarvester with a search source to collect email addresses linked to the domain.
7 Use Netcraft to check the hosting provider and the operating system of the web server.
8 Check the whois record for the administrative and technical contacts, noting that privacy protection may hide them.
9 Use nslookup in interactive mode to query the name server directly and compare the answers with dig.
10 Compile the DNS, mail and network details into one footprint table for the target.

**Result:** The registrar, creation and expiry dates, name servers, mail exchanger records and the hop-by-hop network path to the target were obtained. Email addresses associated with the domain were also collected. The footprinting information was verified and recorded.

# Experiment 3: Session hijacking using various tools

**Aim:** To demonstrate how a valid session identifier can be captured and reused to access a web application without logging in, in a controlled lab environment.

**Tools required:**

- DVWA (Damn Vulnerable Web Application) running in a local VM
- Burp Suite
- Firefox with a cookie editor add-on
- Wireshark
- Ettercap (lab network only)

**Procedure**

1 Start the lab VM hosting DVWA and set the security level to Low from the DVWA Security menu.
2 Log in to DVWA in the first browser as the admin user.
3 Open the browser developer tools or the cookie editor and note the value of the PHPSESSID cookie.
4 Capture the HTTP traffic in Burp Suite or Wireshark while the session is active and confirm that the cookie is sent in every request.
5 Open a second browser or a private window on the same lab machine and set the same PHPSESSID value in its cookie storage.
6 Refresh the DVWA page in the second browser and check whether the application opens the admin session without asking for a password.
7 Log out in the first browser and check whether the second browser session is still accepted.
8 Open the Burp Suite proxy history and locate the request that carries the cookie header.
9 Repeat the test in a new browser profile to confirm that the cookie alone is enough for access.
10 Note how the application behaves after logout to check whether the old session token is invalidated.

**Result:** By copying the session cookie of an active login into a second browser, access to the authenticated DVWA session was obtained without entering the credentials. This showed that a session token alone is enough to impersonate a logged-in user when it is not protected.

# Experiment 4: Detecting session hijacking

**Aim:** To identify the signs of session hijacking and to check the protections that detect or stop it.

**Tools required:**

- Wireshark
- arpwatch
- Burp Suite
- Browser developer tools

**Procedure**

1 Start Wireshark on the lab network interface and apply a display filter such as http.cookie to observe session cookies in plain HTTP traffic.

2 Log in to the web application and note the session ID assigned before and after login.

3 Check whether the session ID changes after a successful login. A session ID that does not change after login is a weakness.

4 Use arpwatch to detect changes in the IP to MAC address mapping that may indicate ARP spoofing on the lab network.

5 Check whether the application binds the session to the IP address or the User-Agent and whether it expires after logout or inactivity.

6 Record the indicators found: repeated session IDs, cookies sent over HTTP, MAC address changes and sessions active from two different hosts.

7 Compare the session ID values across several logins to see whether they are predictable or repeated.

8 Check the Set-Cookie response header for the HttpOnly and Secure attributes and record whether they are present.

9 Review the application or server logs, if available, for logins from unexpected IP addresses.

**Result:** The session identifiers were observed in captured traffic, and the lab application was found to send cookies without the Secure flag. Changes in MAC address mapping were recorded by arpwatch. These indicators can be used to detect a hijacked session, and the protections suggested are session regeneration on login, HTTPS only cookies and short session timeouts.

# Experiment 5: Web server reconnaissance using various tools

**Aim:** To identify the web server software, its version, open ports and the technologies used by the target web server.

**Tools required:**

- nmap
- whatweb
- netcat
- Nikto
- Wappalyzer (browser add-on)

**Procedure**

1 Run nmap -sV -p 80,443 <target IP> to identify the open web ports and the service version running on them.

2 Run whatweb <target URL> to identify the server header, the programming language and the CMS used.

3 Perform banner grabbing with netcat: nc <target IP> 80, then type HEAD / HTTP/1.0 and press Enter twice to read the response headers.

4 Run nikto -h <target URL> to get the server banner and a list of basic server-level findings.

5 Compare the server name and version obtained from each tool and record the output.

6 Run nmap with the -O option, if permitted in the lab, to estimate the operating system of the server.

7 Check the HTTP response for the Server and X-Powered-By headers and record their values.

8 Compare the results of all tools and mark which information was found by more than one tool.

**Result:** The web server software, its version, the open ports (80 and 443) and the technologies behind the site were identified. The response headers obtained by banner grabbing matched the output of nmap and whatweb.

# Experiment 6: Enumerating web server information

**Aim:** To enumerate the directories, HTTP methods, titles and other server details of the target web server.

**Tools required:**

- nmap NSE scripts
- dirb or gobuster
- Nikto
- curl

**Procedure**

1 Run nmap -p 80 --script http-title,http-headers,http-methods <target IP> to read the page title, response headers and allowed HTTP methods.
2 Run nmap -p 80 --script http-enum <target IP> to list common directories and files.
3 Run dirb http://<target IP>/ with the default wordlist to discover hidden directories.
4 Use curl -I http://<target URL> to check the response status codes and headers manually.
5 Run nikto -h http://<target IP> to confirm the directories and risky files found.
6 Note any directory that returns status code 200 or 403, and any HTTP method that is enabled but not needed, such as PUT or TRACE.
7 Run dirb with a different wordlist such as common.txt to find additional directories.
8 Use curl -X OPTIONS to confirm the HTTP methods allowed by the server.
9 Record the status code of each directory found: 200 means accessible, 301 means redirect and 403 means forbidden.

**Result:** Several directories, the page title, server headers and the allowed HTTP methods were enumerated. The enumerated information was recorded and matched between nmap, dirb and Nikto.

# Experiment 7: Web application reconnaissance using various tools

**Aim:** To identify the components, frameworks and entry points of a web application.

**Tools required:**

- Wappalyzer
- WhatWeb
- Burp Suite (Target and Site map)
- OWASP ZAP

**Procedure**

1 Open the target web application in Firefox with Wappalyzer enabled and note the frameworks, JavaScript libraries and CMS shown.

2 Run whatweb -v <target URL> and compare the results with Wappalyzer.

3 Start Burp Suite, configure the browser proxy to 127.0.0.1:8080 and browse all main pages of the application.

4 Open the Target tab and review the site map to list all pages, forms and parameters.

5 Open OWASP ZAP, run the Quick Start spider on the target URL and note the URLs found.

6 List the input fields, login forms and query parameters discovered for further testing.

7 View the page source in the browser to check for JavaScript file names and comments that may reveal hidden paths.

8 Note the version numbers of the frameworks shown by Wappalyzer, if any.

9 Record each form, its method (GET or POST) and its input fields in the table.

**Result:** The technologies, the page structure, the forms and the parameters of the web application were identified. The list of entry points was prepared for later vulnerability testing.

# Experiment 8: Web spidering

**Aim:** To crawl a web application and map all its pages, links and resources automatically.

**Tools required:**

- Burp Suite Spider
- OWASP ZAP Spider
- wget
- HTTrack

**Procedure**

1 Open Burp Suite and add the target URL to the scope under the Target tab.
2 Right-click the target in the site map and select Spider this host.
3 Wait until the spider finishes and open the site map to see the discovered URLs.
4 Run wget --mirror --no-parent <target URL> in a separate directory and check the downloaded files.
5 Compare the number of URLs found by Burp Suite, ZAP and wget and record the differences.
6 Save the final list of URLs for use in the vulnerability scanning experiment.
7 Set the spider depth and the number of threads in the spider options to control how many pages are crawled.
8 Review the Burp Suite Target tab to check for pages that the spider did not reach.
9 Count the unique URLs, including those with parameters, and note the total in the result.

**Result:** The web application was crawled and a map of its pages, links and static resources was obtained. The number of URLs discovered by each spider was recorded and the list was saved for further use.

# Experiment 9: Web application vulnerability scanning

**Aim:** To scan a web application for known vulnerabilities and misconfigurations using automated scanners.

**Tools required:**

- Nikto
- OWASP ZAP (active scan)
- Burp Suite Scanner (Community features)
- OpenVAS or Nessus (optional)

**Procedure**

1 Run nikto -h <target URL> -o nikto_report.txt and save the output.
2 Open OWASP ZAP, enter the target URL and run an Automated Scan.
3 Review the alerts in ZAP, sorted by risk level: High, Medium, Low and Informational.
4 Run the vulnerability scan in OpenVAS or Nessus against the target IP if installed.
5 Verify each reported vulnerability manually in the browser to remove false positives.
6 Prepare a table with the vulnerability name, risk level, affected URL and the recommended fix.
7 Check the ZAP alert details to see the exact URL and parameter where each issue was found.
8 Save the Nikto and ZAP reports in the same folder with clear file names.
9 Compare the number of issues found by each scanner and note any issue reported by only one scanner.

**Result:** The scanners reported missing security headers, outdated software, directory listing and some injection-type issues. The findings were verified manually and recorded in the report table.

# Experiment 10: Cracking FTP credentials using a dictionary attack

**Aim:** To find the password of an FTP account by trying words from a dictionary file.

**Tools required:**

- Hydra
- Medusa or Ncrack
- A wordlist such as rockyou.txt
- Metasploitable 2 VM (FTP service on port 21)

**Procedure**

1 Start the Metasploitable 2 VM and find its IP address using ifconfig.
2 Check that port 21 is open with nmap -p 21 -sV <target IP>.
3 Extract the wordlist with gunzip /usr/share/wordlists/rockyou.txt.gz if it is compressed.
4 Run hydra -l <username> -P /usr/share/wordlists/rockyou.txt ftp://<target IP> -t 4 -V.
5 Wait until Hydra reports a valid login with the password it found.
6 Log in with the recovered credentials using ftp <target IP> to confirm the password.
7 Keep the number of parallel tasks low to avoid locking the account or flooding the service.
8 Check the Hydra output for every valid username and password pair found.
9 Note the number of password attempts made before the valid password was found.

**Result:** The FTP service was attacked with the dictionary file and the valid password for the test account was recovered. The login was confirmed by connecting to the FTP server with the recovered credentials.

# Experiment 11: Performing Cross-site Request Forgery (CSRF) attack

**Aim:** To demonstrate how a logged-in user can be made to submit an unintended request to a web application.

**Tools required:**

- DVWA (CSRF module, security level Low)
- Burp Suite
- A text editor
- Firefox

**Procedure**

1 Log in to DVWA and set the security level to Low.
2 Open the CSRF module and submit a password change once manually while capturing the request in Burp Suite.
3 Note the URL and the parameters used for the password change request.
4 Create a simple HTML file that contains a form with the same action URL and the same parameters, and a script that submits the form automatically.
5 Open the HTML file in the same browser while still logged in to DVWA.
6 Check whether the password was changed without the user entering it in the DVWA page.
7 Confirm that the DVWA security level is still Low before starting the test.
8 Explain in the record that the password change form has no CSRF token, which is why the forged request works.
9 Log out and log in with the new password to confirm that the change was successful.

**Result:** The password of the logged-in DVWA user was changed by the forged request sent from the HTML page. This showed that the application does not verify a CSRF token or the origin of the request.

# Experiment 12: Identifying XSS vulnerabilities in web applications

**Aim:** To identify reflected and stored Cross-site Scripting (XSS) vulnerabilities in a web application.

**Tools required:**

- DVWA (XSS reflected and XSS stored modules)
- Burp Suite
- Browser developer tools

**Procedure**

1 Set the DVWA security level to Low and open the Reflected XSS module.
2 Enter a simple test string such as test123 in the name field and check whether it is printed back in the page.
3 Enter the script test payload <script>alert(1)</script> and observe whether the alert box appears.
4 Open the Stored XSS module, submit a message containing the same test payload and reload the page.
5 Check the page source using the developer tools to see how the input is placed in the HTML.
6 Repeat the test at the Medium security level and note the change in the filtering behaviour.
7 Test a payload that uses an HTML attribute, such as an image tag with an onerror handler, and note whether it runs.
8 Observe that the payload runs because the input is printed back to the page without encoding.
9 Note the change in filtering behaviour at the Medium and High security levels.

**Result:** The reflected and stored XSS payloads were executed in the browser, which confirmed the vulnerability at the Low security level. At the higher level, the filtering changed the output, and the difference was recorded.

# Experiment 13: Detecting web application vulnerabilities using various web application security tools

**Aim:** To detect vulnerabilities in a web application using different security tools and compare their results.

**Tools required:**

- OWASP ZAP
- Burp Suite
- Nikto
- w3af or Wapiti

**Procedure**

1 Start the DVWA lab VM and confirm that the target is reachable from Kali Linux.
2 Run an active scan with OWASP ZAP and note the alerts for SQL injection, XSS, CSRF and missing headers.
3 Run an audit with Burp Suite on the same pages and compare the issues found.
4 Run wapiti -u http://<target URL> to get a second independent scan report.
5 Run nikto -h http://<target URL> and check the server-level findings.
6 Create a comparison table listing each vulnerability, the tools that detected it and its risk level.
7 Save each tool's report in its own file so that the findings can be compared later.
8 Mark each finding as a true positive or a false positive after checking it manually.
9 Prepare a summary of the total vulnerabilities found by each tool.

**Result:** Several vulnerabilities including SQL injection, reflected XSS, CSRF and missing security headers were detected. Each tool detected a different subset of issues, so the combined results gave a more complete picture of the application.

# Experiment 14: Performing an SQL injection attack against MSSQL to extract databases

**Aim:** To extract the names of the databases stored on a Microsoft SQL Server through an SQL injection vulnerability in a lab web application.

**Tools required:**

- A lab web application connected to MSSQL (Windows VM)
- sqlmap
- Burp Suite
- Kali Linux

**Procedure**

1 Start the lab Windows VM with the vulnerable web application and MSSQL server, and confirm the application URL is reachable.
2 Open the page with a numeric parameter such as id=1 and add a single quote to check for a database error.
3 Use Burp Suite to save the request as a file, which is useful input for sqlmap.
4 Run sqlmap -r request.txt --dbms=mssql --batch to confirm the injectable parameter.
5 Run sqlmap -r request.txt --dbms=mssql --dbs to list the databases on the server.
6 Run sqlmap -r request.txt --dbms=mssql -D <database name> --tables to list the tables in the selected database.
7 Record the database names and the tables found in the result table.
8 Compare the page response for a true condition and a false condition to confirm the injection point.
9 Use sqlmap with the --current-db option to confirm the database in use.
10 Save the list of databases in a text file and attach it to the record.

**Result:** The injectable parameter was confirmed and sqlmap listed the databases present on the MSSQL server. The names of the databases and the tables of the selected database were extracted successfully in the lab.

# Experiment 15: Detecting SQL injection vulnerabilities using various SQL injection detection tools

**Aim:** To detect SQL injection vulnerabilities in a web application using automated tools and manual testing.

**Tools required:**

- sqlmap
- OWASP ZAP
- Burp Suite Scanner
- Nikto
- Manual test strings in the browser

**Procedure**

1 Identify the parameters that accept user input, such as the id parameter in the lab application.
2 Enter a single quote and a double quote in the parameter manually and check for SQL error messages in the response.
3 Run sqlmap -u "<URL with parameter>" --batch --level=2 to detect injection automatically.
4 Run an active scan in OWASP ZAP and note the SQL injection alerts with their confidence level.
5 Run an audit in Burp Suite on the same parameter and compare the findings.
6 Record the injection type found (error based, boolean based, time based or union based) for each tool.
7 Test the parameter with a numeric value and a string value and compare the responses.
8 Check the sqlmap output to see which injection technique was detected for the parameter.
9 Record the back-end DBMS version reported by sqlmap and compare it with the lab setup.

**Result:** SQL injection was detected by manual testing, sqlmap, ZAP and Burp Suite on the same parameter. sqlmap reported the injection technique and the back-end DBMS, and the results of all the tools agreed.

# Experiment 16: Footprinting a wireless network

**Aim:** To identify the wireless networks around the lab, their SSIDs, BSSIDs, channels, encryption types and signal strength.

**Tools required:**

- Kali Linux with a wireless adapter that supports monitor mode
- airmon-ng
- airodump-ng
- iwlist (or netsh wlan on Windows)
- Wireshark

**Procedure**

1 Connect the wireless adapter to the Kali Linux VM and check it with iwconfig.
2 Run airmon-ng check kill and then airmon-ng start wlan0 to enable monitor mode.
3 Run airodump-ng wlan0mon to list the nearby access points and note the BSSID, channel, privacy (encryption), cipher, authentication and power level.
4 Run airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon to focus on one access point, only on the lab router.
5 On Windows, run netsh wlan show networks mode=bssid to compare the details.
6 Stop the capture with Ctrl+C and record the findings in a table.
7 Note whether each access point uses WEP, WPA, WPA2 or open authentication.
8 Record the number of clients connected to each access point, if shown by airodump-ng.
9 Stop monitor mode after the scan with airmon-ng stop wlan0mon and restore normal networking.

**Result:** The nearby access points were listed with their SSID, BSSID, channel, encryption type and signal strength. The encryption used by the lab router was identified as WPA2, and the data was recorded for analysis.

# Experiment 17: Creating a rogue access point to capture data packets

**Aim:** To create a fake access point in an isolated lab that captures the packets of client devices connected to it.

**Tools required:**

- Kali Linux with a wireless adapter in monitor mode
- hostapd
- dnsmasq
- airbase-ng
- Wireshark
- Two lab devices (client and attacker) on an isolated network

**Procedure**

1 Use only the lab devices and an isolated network, and make sure no real network is affected.
2 Install and check hostapd and dnsmasq using apt install hostapd dnsmasq if they are not present.
3 Write a basic hostapd configuration file with the interface, a lab SSID, channel and open authentication.
4 Configure dnsmasq to give IP addresses to the connected lab client through DHCP.
5 Start the access point with hostapd and enable IP forwarding using echo 1 > /proc/sys/net/ipv4/ip_forward.
6 Connect the lab client device to the rogue SSID and start Wireshark on the attacker interface.
7 Browse a test page from the client and check the captured packets in Wireshark.
8 Stop all services and restore the network settings after the experiment.
9 Confirm that the lab client received an IP address from dnsmasq.
10 Filter the captured packets in Wireshark with the http filter and note the visible content.
11 Explain in the result that a rogue access point can read unencrypted traffic, and that HTTPS protects the data.

**Result:** The rogue access point was created, the lab client connected to it and received an IP address from dnsmasq. Packets from the client were captured in Wireshark and the captured HTTP traffic was analysed. The experiment showed why clients should use encrypted connections such as HTTPS and WPA2 Enterprise.